JOMHACK
EMPOWERING INNOVATION

```
> bootcamp.learn()
```

JOMHACK
EMPOWERING INNOVATION

JomCode Coding Bootcamp

CHAPTER 07

Functions, parameters, return values and scope

`learn / type / run / explain`

Plain meaning and technical explanation

PLAIN MEANING

A function gives a reusable name to a small job.

TECHNICAL EXPLANATION

def defines a function. Parameters receive arguments. return sends a result to its caller. Local names belong to a function call.

REAL-WORLD USE

Reuse title normalization, previews and validation rules.

QUICK TIP

Returning a value lets callers decide whether to display or store it.

INSTALL / RUN

No packages. Windows: py workbook.py. macOS: python3 workbook.py.

Worked example 1: A reusable preview

workbook.py

```
01    def preview(text, limit=5):
02        return text.strip()[:limit]
03
04    print(preview(" Hello world "))
05    print(preview(" Hello world ", limit=8))
```

PRO TIP Keyword arguments make a call easier to understand.

Run and explain example 1

EXPECTED RESULT

Hello
Hello wo

WHY IT BEHAVES THIS WAY

The default argument is used when the caller omits limit.

Worked example 2: Return several results

workbook.py

```
01    def clean_fields(title, body):
02        return title.strip(), body.strip()
03
04    title, body = clean_fields(" Hi ", " Welcome ")
05    print(title)
06    print(body)
```

PRO TIP Keep input cleaning separate from database writes.

Run and explain example 2

EXPECTED RESULT

Hi
Welcome

WHY IT BEHAVES THIS WAY

A tuple carries both results and unpacking assigns them to names.

Worked example 3: Local scope

workbook.py

```
01    title = "Outside"
02    def label():
03        title = "Inside"
04        return title
05
06    print(label())
07    print(title)
```

PRO TIP Pass data into functions instead of relying on mutable globals.

Run and explain example 3

EXPECTED RESULT

Inside
Outside

WHY IT BEHAVES THIS WAY

The local title inside label() does not reassign the outer title.

Worked example 4: Guard clauses

workbook.py

```
01    def can_edit(user_id, author_id):
02        if user_id is None:
03            return False
04        return user_id == author_id
05
06    print(can_edit(None, "a"))
07    print(can_edit("a", "a"))
```

PRO TIP A function without an explicit return returns None.

Run and explain example 4

EXPECTED RESULT

False
True

WHY IT BEHAVES THIS WAY

Returning early handles a special case before the main rule.

Exercise 1: Title validator

YOUR TASK

Write valid_title(text) that checks the trimmed length is between 1 and 160.

CHECK THESE CASES

Whitespace fails. 160 characters succeeds. 161 fails.

HINT AFTER A TRY

Return a chained comparison.

Exercise 2: Safe list default

YOUR TASK

Write add_title(title, titles=None), returning a new list when none is passed.

CHECK THESE CASES

Two calls without a list do not share items. A supplied list receives the item.

HINT AFTER A TRY

Create [] inside the function.

Checkpoint before moving on

01 Run and explain all four worked examples. Match their actual results to the expected behavior.

02 Attempt both exercises before reading the solutions. Verify every acceptance case.

03 Explain how this chapter supports Postlight: Reuse title normalization, previews and validation rules.

Official reference